

D2

Threat Intelligence Correlation & Alert Prioritisation Assistant

AI Build Specification for Coding Agents

PROJECT GOAL

Build an AI system that reads security alerts from multiple sources, connects related alerts, separates real threats from false positives, maps attacker behaviour to MITRE ATT&CK, ranks threats by priority, and creates short BLUF summaries for decision makers.

Suggested Product Name: SentinelAI

Tagline: From thousands of alerts to one clear decision.

Source basis: D2 challenge in the uploaded Industry Problem Statements 2026 document. This specification expands the required D2 features into a build-ready MVP plan.

1. Original D2 Problem - Simple Meaning

Security teams receive a very large number of alerts every day. These alerts can come from SIEM tools, cyber sensors, satellite feeds, and intelligence reports. They are not always in the same format. A human team cannot manually read and investigate every alert quickly enough.

The required D2 outcome

The system must ingest multi-source threat feeds, correlate alerts, separate genuine threats from false positives, map attacker techniques to MITRE ATT&CK, and generate prioritised BLUF investigation summaries.

What the final product should do

- Collect or import alerts from multiple sources.
- Convert different alert formats into one common structure.
- Find alerts that are related to the same incident.
- Decide whether an incident is probably a real threat or a false positive.
- Give each incident a severity, confidence score, and priority.
- Map suspicious behaviour to MITRE ATT&CK techniques.
- Generate a short BLUF summary that tells the important result first.
- Show the highest-risk incidents at the top of the dashboard.

2. Required Features - Must Build

ID	Feature	What it does	Priority
F1	Multi-source threat feed input	Support demo inputs for SIEM alerts, cyber sensor alerts, satellite-style alerts, and intelligence reports. For the MVP, these can be simulated JSON, CSV, or plain text.	MUST
F2	Data normalisation	Convert every incoming alert into one common format: alert ID, source, timestamp, user/device, IP, event, description, raw severity, and metadata.	MUST
F3	Alert correlation	Group alerts that may belong to the same incident. Use matching signals such as same IP, same user, same device, similar event, and close timestamps.	MUST
F4	Genuine threat detection	Classify an incident as likely genuine when the combined evidence looks dangerous.	MUST

F5	False-positive detection	Classify alerts/incidents that are probably harmless and explain why they look like false positives.	MUST
F6	Threat prioritisation	Assign priority such as Critical, High, Medium, Low, or False Positive. Sort Critical incidents first.	MUST
F7	MITRE ATT&CK mapping	Map detected behaviour to one or more MITRE ATT&CK technique IDs and names.	MUST
F8	BLUF summary generation	Generate a short Bottom Line Up Front report with situation, risk, evidence, and immediate recommended action.	MUST
F9	Prioritised investigation view	Show ranked incidents with severity, score, confidence, number of related alerts, and current status.	MUST

3. Recommended AI Workflow

Do not make one giant prompt do everything.

Use a simple pipeline. Normal code handles matching and scoring rules. The LLM handles understanding, explanation, MITRE suggestions, BLUF writing, and chat answers.

1. Ingest alert -> accept JSON, CSV, or text.
2. Normalize -> convert to a standard alert object.
3. Correlate -> group related alerts into one incident.
4. Analyze -> ask the LLM what is probably happening.
5. Classify -> Genuine Threat or False Positive.
6. Prioritize -> produce severity, threat score, and confidence.
7. MITRE Map -> return technique ID and technique name.
8. Generate BLUF -> short commander-ready summary.
9. Store result -> make it available to the dashboard and AI Copilot.

Suggested logical agents

Correlation Agent: Finds related alerts and creates one incident.

Threat Analysis Agent: Explains what the combined alerts may mean.

MITRE Mapping Agent: Maps behaviour to relevant MITRE ATT&CK techniques.

Risk Agent: Produces severity, score, confidence, and genuine/false-positive classification.

BLUF Report Agent: Creates a short executive summary and recommended next action.

4. User Interface - Required Screens

Screen 1 - Command Center

KPI cards: total alerts, correlated incidents, Critical, High, Medium, Low, False Positive. Show a ranked incident table with Critical items first.

Screen 2 - Threat Feed

Show incoming alerts with source, time, event, raw severity, and processing status. Include filters by source and severity.

Screen 3 - Incident Analysis

Show correlated alerts, attack timeline, threat score, confidence, AI explanation, genuine/false-positive result, MITRE mapping, BLUF summary, and recommended actions.

Screen 4 - AI Copilot

Chat interface for questions such as: What is the highest-risk threat? Why is incident #1042 critical? What should we investigate first?

Suggested Command Center layout

Top area Total Alerts Active Incidents Critical High False Positives
Main table Priority Incident ID Title Severity Threat Score Confidence Related Alerts Status
Side panel AI Copilot + quick question buttons

5. Data Model for the MVP

Normalized alert object

```
{
  "alert_id": "A1042",
  "source": "firewall",
  "timestamp": "2026-09-14T02:13:00",
  "user": "admin",
  "device": "server-01",
  "ip": "103.44.17.9",
  "event": "multiple_failed_logins",
  "description": "23 failed login attempts",
  "raw_severity": "medium"
}
```

Incident object

```
{
  "incident_id": "INC-1042",
  "title": "Possible administrator account compromise",
  "related_alert_ids": ["A1042", "A1043", "A1044"],
  "classification": "Genuine Threat",
  "severity": "Critical",
  "threat_score": 96,
  "confidence": 94,
  "mitre": [
    {"id": "T1110", "name": "Brute Force"},
    {"id": "T1078", "name": "Valid Accounts"}
  ],
  "bluf": "Possible compromised admin account...",
  "recommended_actions": ["Disable account", "Block suspicious IP"]
}
```

6. Correlation Rules for Demo

For the hackathon MVP, use simple deterministic rules first. The LLM can explain the result, but matching should stay predictable.

- Same user within 30 minutes -> add correlation points.
- Same IP within 30 minutes -> add correlation points.
- Same device/host -> add correlation points.
- Failed login followed by successful login -> strong relationship.
- Login followed by large data download -> strong relationship.
- Same malware hash/domain/IOC -> strong relationship.
- If total correlation score crosses a threshold, group alerts into one incident.

Example sequence

02:13 - Firewall 23 failed login attempts against admin account
02:17 - Authentication Successful login from the same IP
02:23 - File Server 4.2 GB downloaded by the admin account
AI conclusion These alerts are likely related and may represent an account compromise followed by suspicious data access.

7. Threat Scoring for the MVP

Use an easy-to-explain score from 0 to 100. The exact values are demo logic, not a military standard.

Signal	Example Score
Admin or privileged account	+20
Many failed logins	+15
Unknown or unusual source IP	+15
Successful login after failures	+15
Large or unusual data transfer	+20
Malware/known malicious IOC	+25
Multiple correlated sources	+10

Suggested severity ranges: 85-100 Critical, 70-84 High, 45-69 Medium, 20-44 Low, and below 20 can be considered low-value or possible false-positive depending on context.

8. BLUF Output Format

BLUF template

BOTTOM LINE: [one-sentence conclusion]

SEVERITY: [Critical/High/Medium/Low]

CONFIDENCE: [0-100%]

KEY EVIDENCE: [2-4 short points]

MITRE ATT&CK: [technique IDs + names]

IMMEDIATE ACTION: [first recommended response]

NEXT INVESTIGATION: [what analyst should check next]

Example BLUF

BOTTOM LINE: A possible compromised administrator account accessed internal systems and performed an unusual large download.

SEVERITY: Critical

CONFIDENCE: 94%

KEY EVIDENCE: 23 failed logins; successful login from the same IP; 4.2 GB download.

MITRE ATT&CK: T1110 Brute Force; T1078 Valid Accounts.

IMMEDIATE ACTION: Disable the account and isolate the affected endpoint.

NEXT INVESTIGATION: Review downloaded files and related host activity.

9. AI Prompt Contracts

Threat analysis prompt - required output

Ask the model to return JSON only. Required fields: title, classification, severity, threat_score, confidence, explanation, evidence, mitre_techniques, bluf, recommended_actions. Tell the model to use only the supplied alert data and clearly say when evidence is insufficient.

Copilot prompt rules

- Answer only from incidents and alerts available in the project database.
- Never invent an alert, IP, user, or event that is not in the supplied data.
- Always mention the incident ID when discussing a specific incident.
- Keep commander answers short; give deeper details only when asked.
- If the available evidence is weak, say confidence is low instead of pretending certainty.

10. Suggested Technical Architecture

Frontend React or Next.js + Tailwind CSS + Recharts + Lucide icons
Backend Python FastAPI. Keep endpoints simple and JSON based.
Storage SQLite for incidents/alerts. JSON seed data is acceptable for the first demo.
AI Any available LLM API that can return structured JSON. Use one model with different system prompts if multiple models are not available.
MITRE data Store a small curated JSON list of common MITRE ATT&CK technique IDs and descriptions for demo mapping.

Suggested API endpoints

Endpoint	Purpose
POST /alerts/import	Import one or many alerts.
GET /alerts	List normalized alerts.
POST /incidents/correlate	Run correlation and create/update incidents.
POST /incidents/{id}/analyze	Run AI analysis for one incident.
GET /incidents	Return ranked incidents.
GET /incidents/{id}	Return full incident detail.
POST /copilot/chat	Ask questions about stored incidents.
GET /dashboard/summary	Return counts for KPI cards and charts.

11. Sample Demo Dataset - Minimum

Create at least 20-30 alerts containing a mix of normal activity, false positives, and 3-5 real incident stories. This makes the dashboard look alive without requiring a real security system.

- Incident A: brute-force attempts -> successful login -> data download.
- Incident B: suspicious PowerShell/process event -> outbound connection.
- Incident C: known malicious IP contacted by two internal devices.
- False Positive A: employee enters wrong password three times.
- False Positive B: approved vulnerability scanner creates many network alerts.

12. Acceptance Criteria - AI Should Not Stop Until These Work

- ☐ User can load demo threat alerts from at least three different source types.
- ☐ Alerts are displayed in one normalized format.

- ☐ Related alerts are grouped into an incident.
- ☐ Each incident receives a Genuine Threat or False Positive result.
- ☐ Each incident has severity, threat score, and confidence.
- ☐ At least one incident is mapped to valid MITRE ATT&CK technique IDs and names.
- ☐ Every analyzed incident has a BLUF summary.
- ☐ Incident list is sorted by priority with Critical first.
- ☐ Dashboard counts match the stored incident data.
- ☐ AI Copilot can answer questions using the current incident data.
- ☐ No page should depend on fake hard-coded output that disagrees with the stored data.
- ☐ The full demo works from input -> correlation -> analysis -> prioritized report.

13. Build Order for the Coding AI

10. Create project structure and seed data.
11. Create backend data models and import endpoint.
12. Create normalization logic.
13. Create correlation engine with deterministic rules.
14. Create threat scoring and severity logic.
15. Connect LLM and enforce structured JSON output.
16. Add MITRE mapping using a local technique list + LLM selection.
17. Generate BLUF and recommended actions.
18. Build dashboard and incident detail page.
19. Build threat feed.
20. Build AI Copilot.
21. Polish UI, add charts, loading states, and error handling.
22. Test the complete demo story.

14. Final Demo Story

23. Open Command Center and show that many alerts were received.
24. Explain that SentinelAI reduced them into a smaller number of correlated incidents.
25. Open the highest-risk incident.
26. Show the three related alerts and attack timeline.
27. Show Genuine Threat classification, Critical severity, score, and confidence.
28. Show MITRE ATT&CK mapping.
29. Generate or reveal the BLUF summary.
30. Ask the Copilot: "What should we investigate first and why?"
31. End with the message: "From thousands of alerts to one clear decision."

15. Nice-to-Have Features - Only After MVP Works

- Upload CSV/JSON from the UI.
- Live alert simulation every few seconds.
- Attack timeline visualization.
- Graph showing connected users, IPs, devices, and incidents.
- PDF export for BLUF reports.
- Dark cyber-security theme and subtle animations.
- Filters, search, and incident status workflow.

- Analyst feedback buttons: Confirm Threat / Mark False Positive.

Important scope rule

First complete every MUST feature. Do not spend hackathon time on advanced animations, complex authentication, real network integrations, or large-scale ML training until the required D2 workflow works end to end.

16. Copy-Paste Master Instruction for an AI Coding Agent

MASTER PROMPT

Build a complete hackathon MVP named SentinelAI for the D2 problem: Threat Intelligence Correlation & Alert Prioritisation Assistant. Use React/Next.js + Tailwind for the frontend and FastAPI + SQLite for the backend. The system must ingest simulated multi-source security alerts, normalize them, correlate related alerts into incidents, separate likely genuine threats from false positives, prioritize incidents by severity and score, map attacker behaviour to MITRE ATT&CK techniques, and generate structured BLUF summaries. Include four main screens: Command Center, Threat Feed, Incident Analysis, and AI Copilot. Use deterministic rules for alert correlation and basic scoring, then use an LLM for incident explanation, classification support, MITRE suggestions, BLUF writing, recommended actions, and Copilot Q&A. Seed the app with 20-30 realistic demo alerts and at least 3 real incident stories plus false positives. Make all AI outputs structured JSON. The UI must be polished, responsive, and demo-ready. Build feature by feature in the order defined in this specification and verify each acceptance criterion before moving to optional features.